

Lesson 2 — Vectors For Real

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

The recipe

Subtract, normalise, scale.

```
const toTarget = target.sub(me); //
1. which way?
const direction = toTarget.normalise(); //
2. direction only
me = me.add(direction.scale(SPEED*dt)); //
3. how much
```

Say step 1 out loud: **target minus me.**

The whole class

```
add(o)    → (x+o.x, y+o.y)
sub(o)    → (x-o.x, y-o.y)
scale(k)  → (x*k, y*k)
length()  → Math.hypot(x, y)
normalise() → divide both by length
dot(o)    → x*o.x + y*o.y
angle()   → Math.atan2(y, x) ← y FIRST
Vec2.fromAngle(r) → (cos r, sin r)
```

Normalise

```
(30, 40) → (0.6, 0.8)
length 50    length 1
```

A length-1 vector is a **unit vector**: a pure direction with the distance removed.

The guard you must never remove

```
normalise() {
  const len = this.length();
  if (len === 0) return new Vec2(0, 0);
  return new Vec2(this.x/len, this.y/len);
}
```

Without it: `0/0 = NaN`, which spreads to everything it touches, never throws, and never recovers. A sprite silently vanishes for ever.

Four behaviours, one line apart

```
chase: (target - me).normalise()
flee:  (me - target).normalise()
orbit: perpendicular of chase → (-y, x)
cone:  facing.dot(toTarget) > 0.7
```

Angles

- Angle **o** points **right**, not up.
- Angles grow **clockwise**, because y points down.
- Radians, not degrees. A full turn is $2\pi \approx 6.283$.
- `atan2(y, x) → angle · cos / sin → direction.`

Dot product of two unit vectors

value	meaning
1	same way
0	right angles
-1	opposite

Drawing something rotated

```
ctx.save();
ctx.translate(pos.x, pos.y);
ctx.rotate(angle);
drawShape(ctx); // as if at (0,0)
ctx.restore(); // NEVER forget
```

Section A — Recall

A1. What does it mean to *normalise* a vector, and what is the result called? [2 marks]

A2. Write the sentence that tells you which way round a subtraction goes when you want the direction from A to B. [2 marks]

A3. Why must `normalise()` check for a length of zero? Name the value you get without the check, and say why it is harder to find than a crash. [3 marks]

A4. The same two numbers `(100, 50)` can mean two different kinds of thing. What are they, and which one should you never add to another of its own kind? [2 marks]

A5. Which direction does angle 0 point in, and which way do angles grow on a canvas? Why? [2 marks]

A6. For two unit vectors, what do dot products of 1, 0 and -1 tell you? [3 marks]

A7. When would you use `lengthSquared()` instead of `length()`, and why is it allowed? [2 marks]

Section B — Predict the output

Write your answer down before you run anything.

B1. What are the length and the normalised form of `(9, 12)`? Show your working. [3 marks]

B2. An enemy is at `(100, 100)`, the player at `(160, 180)`, `SPEED` is 200 and `dt` is 0.1. Where is the enemy after one frame of the recipe? Give the numbers. [4 marks]

B3. The `.normalise()` is deleted from B2. Where does the enemy end up after that one frame, and what is wrong with it as a behaviour? [3 marks]

B4. What does `Vec2.fromAngle(0)` return? And `Vec2.fromAngle(Math.PI / 2)`? Say where each one points *on the screen*. [3 marks]

B5. A guard faces `(1, 0)`. The player is at 30° to its right, then at 60° , then at 120° . The test is `dot > 0.7`. Which of the three are seen? (`cos 30 = 0.87`, `cos 60 = 0.5`, `cos 120 = -0.5`.) [4 marks]

Section C — Find and fix the bug

C1. The enemy runs away instead of chasing. One thing is wrong. What? [3 marks]

```
const direction = enemy.pos.sub(player.pos).normalise();
enemy.pos = enemy.pos.add(direction.scale(SPEED * dt));
```

C2. The enemy reaches the player and then vanishes. It never comes back, and the console is empty. Explain exactly what happened, and give the fix. [4 marks]

```
normalise() {
  const len = this.length();
  return new Vec2(this.x / len, this.y / len);
}
```

C3. The ship is drawn at 90° to the direction it travels in. The movement is correct. Two possible causes — name both. [3 marks]

```
const nose = Vec2.fromAngle(ship.angle);
ship.pos = ship.pos.add(nose.scale(SPEED * dt));
// ... later, in draw():
ctx.rotate(Math.atan2(nose.x, nose.y));
```

C4. After about ten seconds the whole game is spinning, and it gets worse the longer it runs. What is missing? [3 marks]

```
ctx.save();
ctx.translate(ship.pos.x, ship.pos.y);
ctx.rotate(ship.angle);
drawShip(ctx);
```

C5. The enemy creeps along at about a pixel a second. The author says “but `SPEED` is 250!”. What has gone wrong, and what is a unit vector's length in pixels? [4 marks]

```
const direction = player.pos.sub(enemy.pos).normalise();
enemy.pos = enemy.pos.add(direction);
```

Section D — Build it

Start from `code/01-the-diagonal-bug.html`, or from your own lesson 1 split.

- ☐ **Checkpoint 1.** Write `vec2.js` yourself: `add`, `sub`, `scale`, `length`, `normalise`. Type it; do not copy ours. Test each method by printing the answer for `(3, 4)` and checking it by hand.
- ☐ **Checkpoint 2.** Fix the diagonal bug in your own movement code using normalised intent. Prove it with a speed readout — the number must be the same in all eight directions.
- ☐ **Checkpoint 3.** Make a ship with an `angle`. `←` and `→` turn it. Draw it with `save` / `translate` / `rotate` / `restore`, and draw a dashed line out of its nose so you can see the direction it will travel.
- ☐ **Checkpoint 4.** `↑` thrusts: move along `Vec2.fromAngle(ship.angle)`. Add screen wrapping.
- ☐ **Checkpoint 5.** `Space` fires. A bullet starts at the nose and its velocity is the nose direction scaled by `BULLET_SPEED`. Give bullets a `life` in seconds and remove them **backwards** through the array.
- ☐ **Checkpoint 6.** Add rocks that drift in random *directions* — pick a random angle, not random x and y speeds. Then say why those two are not the same thing.
- ☐ **Checkpoint 7.** Bullets destroy rocks, using `lengthSquared()` and no square root. Score it.
- ☐ **Checkpoint 8.** Add three chasers that home in on the ship. Make one of them flee instead, by changing exactly one thing.

IF SOMETHING VANISHES

Check for `NaN` first, before anything else. Print the position: `NaN` prints as `NaN`, and once a position is `NaN` it will never be a number again. Something normalised a zero-length vector.

Section E — Design challenge

NO RIGHT ANSWER

Marked on reasoning, and on naming the cost of your idea.

E1. Our `normalise()` returns `(0, 0)` for a zero vector. Three alternatives: throw an error, return `null`, return `(1, 0)`. Argue for one. Which is most likely to **hide** a bug, and which is most likely to be irritating in practice?

E2. Design a homing missile that *turns* towards its target rather than snapping to face it, so it can overshoot, loop round and come back. What does it need to remember beyond a position? What limits how fast it turns? Write the update in words, then in code.

E3. Every `Vec2` method makes a new object. A hundred chasers at 60 fps is about 18,000 new objects a second. Is that a problem? Say how you would **find out** rather than guess, and write down your prediction now so you can check it in lesson 9.

E4. The hard one. A ball bounces off a flat wall by flipping one speed component. Now the wall is at 30°deg;. You have a unit vector along the wall, a unit vector at right angles to it, and the dot product. What does "the part of my velocity pointing into the wall" mean, and what would you do to it?

Stretch goals

1. Add `rotate(radians)` to `Vec2` using $x' = x \cdot \cos - y \cdot \sin$, $y' = x \cdot \sin + y \cdot \cos$.
2. Make a chaser that turns at most 180°deg; per second instead of snapping.

3. Add `lerp` and use it for the camera, a health bar and a menu item.
4. Time ten million `Vec2` creations with `performance.now()`, then compare a mutable `addInPlace`. Report real numbers.
5. Bounce a ball off a slope you can drag (question E4, for real).